Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «КиСБ»
Тема: «Арифметика точечных групп эллиптических кривых, протокол ECDH и алгоритм ECDSA»

Выполнил:
Студент 4-го курса
Группы АС-66
Езепчук А.С.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: программно реализовать арифметику эллиптической кривой над конечным полем, протокол согласования ключей ECDH и электронную цифровую подпись ECDSA; исследовать инверсию по малой теореме Ферма.

Вариант 3

Задание:

1. Реализовать сложение, удвоение и скалярное умножение точек кривой Вейерштрасса в аффинных координатах.

2. Реализовать генерацию ключей, получение общего секрета ECDH, формирование и проверку ECDSA.

3. Реализовать модульную инверсию a^(m−2) с фиксированной структурой вычислений и проанализировать ограничения constant-time в Python.

4. Проверить корректные и ошибочные входы; показать отбраковку подписи после изменения одного байта s.

Теоретическая часть

Конечное поле GF(p) содержит целые числа от 0 до p − 1. Сложение, вычитание и умножение выполняются по модулю p. Деление на ненулевой элемент a заменяется умножением на a^{-1}, для которого $a \cdot a^{-1} \equiv 1 \pmod p$. В программе большие числа представлены встроенным типом int.

Эллиптическая кривая задаётся уравнением $y^2 \equiv x^3 + ax + b \pmod p$. Условие невырожденности: $4a^3 + 27b^2 \not\equiv 0 \pmod p$. Множество точек дополняется бесконечно удалённой точкой O — нейтральным элементом. Для P = (x, y) противоположная точка равна −P = (x, −y mod p).

Используется secp256k1: a = 0, b = 7, cofactor h = 1. Модуль p задаёт поле координат, q — порядок базовой точки G. Эти модули применяются к разным операциям: координатам — p, скалярам ECDSA — q.

```
p = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
q = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
```

Сложение и удвоение точек

Для разных точек P_1 и P_2 с $x_1 \neq x_2$ наклон секущей равен $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \bmod p$. Подстановка прямой $y = \lambda x + \nu$ в уравнение кривой даёт $x^3 - \lambda^2 x^2 + (a - 2\lambda\nu)x + b - \nu^2 = 0$. По теореме Виета сумма трёх абсцисс пересечения равна λ^2.

Отражая третью точку пересечения относительно оси x, получаем сумму $P_3 = P_1 + P_2$: $x_3 = \lambda^2 - x_1 - x_2 \bmod p$; $y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$.

При $P_1 = P_2$ используется касательная. Из $2y \cdot dy/dx = 3x^2 + a$ следует $\lambda = (3x_1^2 + a)(2y_1)^{-1} \bmod p$. Тогда $x_3 = \lambda^2 - 2x_1 \bmod p$, $y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$. Особые случаи: $P + O = P$; $P + (-P) = O$; удвоение при $y = 0$ даёт O.

Скалярное умножение и протокол ECDH

Скалярное умножение kP вычисляет сумму k экземпляров P. Double-and-Add удваивает аккумулятор на каждом бите и прибавляет P при бите 1. В реализации также есть лестница с 256 итерациями: на каждой выполняются одно сложение и одно удвоение, а регистры выбираются маскированным CSWAP.

Алиса выбирает секрет dA и публикует QA = dA·G. Боб выбирает dB и публикует QB = dB·G. Общая точка Алисы dA·QB совпадает с точкой Боба dB·QA, поскольку обе равны (dA·dB)G. Возвращается координата x общей точки в виде 32 байт.

Открытый ключ проверяется до вычисления секрета: координаты должны принадлежать полю, точка должна удовлетворять уравнению кривой, отличаться от O и выполнять qQ = O. Секретный ключ должен удовлетворять $0 < d < q$.

Структура программы

Весь код расположен в lab4_variant3.py. Модуль алгебры представлен Point и Curve; модуль протоколов — Protocols; модуль инверсии — Protection. Дополнительно реализованы hex-интерфейс и встроенная функция selftest. Форматы PEM, DER, ASN.1 и X.509 не используются.

Point — неизменяемая dataclass со slots, полями x, y и infinity. Вложенных динамических контейнеров нет. Однако slots не делает координаты непрерывным статическим блоком: int остаётся объектом Python в динамической памяти. Полное выполнение требования статического хранения средствами этой версии не обеспечивается.

Алгоритм ECDSA

Хэш e = SHA-256(M) интерпретируется как целое число с порядком байтов big-endian. Порядок q имеет длину 256 бит, поэтому дополнительное усечение 256-битного хэша не требуется. Все операции подписи над скалярами выполняются по модулю q.

Формирование подписи: выбрать случайное k, $0 < k < q$; вычислить $C = kG$ и $r = xC \bmod q$. Если $r = 0$, выбрать новое k. Вычислить $s = k^{-1}(e + rd) \bmod q$; при $s = 0$ повторить. Результат — пара (r, s). Для генерации d и k используется secrets.randbelow(q − 1) + 1.

Проверка подписи: проверить диапазоны r и s, допустимость Q. Вычислить $w = s^{-1} \bmod q$, $u_1 = ew \bmod q$, $u_2 = rw \bmod q$ и $C' = u_1G + u_2Q$. Если $C' = O$, подпись отвергается. Иначе она принимается при $xC' \bmod q = r$.

Доказательство корректности: $Q = dG$, поэтому $C' = (ew + rdw)G = (e + rd)s^{-1}G$. Из $s = k^{-1}(e + rd)$ следует $(e + rd)s^{-1} \equiv k \pmod q$. Следовательно, $C' = kG = C$ и $xC' \bmod q = r$.

Инверсия по малой теореме Ферма

Если m — простое и $0 < a < m$, то a^(m−1) ≡ 1 (mod m), следовательно $a^{-1} \equiv$ a^(m−2) (mod m). В Protection.inverse используется m = p при вычислении наклона и m = q при обращении k и s. Показатель m − 2 — публичная константа.

```
exponent = modulus - 2
result = 1
for i in range(modulus.bit_length() - 1, -1, -1):
    square = result * result % modulus
    product = square * a % modulus
    mask = -((exponent >> i) & 1)
    result = square ^ ((square ^ product) & mask)
```

Для p и q выполняется ровно 256 итераций и по два модульных умножения на каждой. Если бит показателя равен 0, маска равна 0 и выбирается square; если бит равен 1, маска равна −1 и выбирается product. Проверка диапазона a выполняется до цикла.

Фиксированная структура алгоритма уменьшает зависимость числа операций от входа. Но Python не гарантирует constant-time для int, умножения, остатка, выделения памяти и побитовых операций. Особые случаи сложения точек и повторный выбор k также содержат ветвления. Поэтому строгая защита всего протокола от timing, SPA и DPA не заявляется.

Запуск программы

Требуется Python 3.10 или новее. Используется только стандартная библиотека: dataclasses, hashlib, secrets и sys. Установка сторонних пакетов не нужна. Данные вводятся как сырые hex-строки без 0x; длина координаты или скаляра — от 1 до 64 цифр.

```
python lab4_variant3.py --test
python lab4_variant3.py keygen
python lab4_variant3.py pub 123456789ABCDEF
python lab4_variant3.py sign 123456789ABCDEF 616263
python lab4_variant3.py verify Qx Qy 616263 r s
python lab4_variant3.py ecdh d Qx Qy
```

В командах verify и ecdh обозначения Qx, Qy, r, s и d заменяются фактическими hex-значениями. Команда pub выводит координаты Q, sign — компоненты подписи. verify печатает valid=1 или valid=0. Отказ подписи — штатный результат; ошибки ввода завершают программу с кодом 1.

Результаты тестирования

Выполнена команда python lab4_variant3.py --test. Получен результат ALL TESTS PASSED: 49, код завершения 0. Проверены групповые операции, обе реализации скалярного умножения, инверсии по p и q, обмен ECDH и цифровые подписи.

Лестница сопоставлена с Double-and-Add для скаляров 1–12. Восемь проверок инверсии выполнены для значений 1, 2, 3 и m − 1 по каждому из двух модулей; результат совпал с pow(a, −1, m). Обращение нуля отвергнуто. Удвоение G совпало с контрольными координатами 2G.

В тесте ECDH заданы dA = 123456789ABCDEF и dB = FEDCBA987654321. Полученные секреты совпали между собой и с результатом версии на C++. Эти ключи являются открытыми учебными значениями; команда keygen генерирует новые случайные ключи.

```
Shared_A =
D8EB786721FF3735E2E64B90EB05B720AA7A65EECC93E22BAB62D13580565C78
Shared_B =
D8EB786721FF3735E2E64B90EB05B720AA7A65EECC93E22BAB62D13580565C78
```

Проверены пустое сообщение, 616263 (abc) и 4C616234205661726961E742033 (Lab4 Variant 3). Во всех случаях хэши совпали с заранее заданными значениями, подпись принята, а искажённая подпись, изменённое сообщение и чужой открытый ключ отвергнуты.

Проверка искажения подписи

Для сообщения abc получены следующие значения одного фактического запуска. При повторном запуске r и s меняются, поскольку каждый раз выбирается новое случайное k.

```
hash =
BA7816BF8F01CFEA414140DE5DAE2223B00361A396177A9CB410FF61F20015AD
r =
F7D9B1058A3DF71A191E64F4EFD414975AF7F8D69F4C622617BA06E23CE27255
s =
5F8497C21030FF4E54A8D5EE67790BA883B43C591D4994A0A31BFE6B7158E902
s_changed =
5F8497C21030FF4E54A8D5EE67790BA883B43C591D4994A0A31BFE6B7158E903
```

В тесте s переводится в 32-байтовый массив; последний байт изменяется операцией XOR 01. Проверка исходной пары возвращает True, после изменения — False. Таким образом, отбраковка показана на конкретном изменении одного байта параметра s.

Дополнительно отвергнуты r = 0, s = q, открытый ключ O и точка вне кривой. Проверена допустимость открытого ключа, полученного из случайного секретного ключа.

Вывод

Реализованы арифметика secp256k1 в аффинных координатах, генерация ключей, ECDH, ECDSA с SHA-256 и инверсия по малой теореме Ферма. Весь исполняемый код размещён в одном Python-файле с тремя логическими модулями. Корректность подтверждена 49 встроенными проверками, контрольными значениями и отбраковкой искажённой подписи.

Реализация учебная. Инверсия имеет фиксированное число шагов, однако строгие constant-time гарантии и непрерывное статическое размещение больших чисел чистым Python не обеспечиваются. Сырой секрет ECDH требует KDF и аутентификации перед прикладным использованием; протокол сам по себе не защищает от подмены участников.

Источники

1. Задание лабораторной работы №4, вариант 3.

2. Исходный код lab4_variant3.py и результаты встроенного selftest.

3. Python Standard Library: hashlib, secrets, dataclasses. Справочная документация используемых модулей.

Листинг программы

Полный исходный код файла lab4_variant3.py приведён ниже.

```
#!/usr/bin/env python3
"""Лабораторная 4, вариант 3. Python 3.10+, без сторонних пакетов.

Запуск тестов: python lab4_variant3.py
Команды: keygen; pub D; ecdh D QX QY; sign D MESSAGE_HEX;
         verify QX QY MESSAGE_HEX R S
Все данные вводятся hex без 0x. Для пустого сообщения передать "".

Модули логически разделены, весь код находится в одном файле.
ВАЖНО: фиксированное число шагов инверсии не делает Python int
constant-time. Арифметика интерпретатора, выделение памяти и особые
случаи точек могут зависеть от данных. Это учебная реализация алгоритма
Ферма, не эквивалент защищённому native-коду предыдущей версии.
Вложенных динамических контейнеров нет. Полностью статическое хранение
BigInt средствами чистого Python невозможно.
"""
from dataclasses import dataclass
import hashlib
import secrets
import sys

# МОДУЛЬ 3. Инверсия через малую теорему Ферма.
class Protection:
    @staticmethod
    def inverse(a: int, modulus: int) -> int:
        """modulus — фиксированное доверенное простое число p или q.

        Ровно 256 итераций и 2 умножения на итерацию для secp256k1.
        Выбор результата — арифметической маской без if по биту a.
        Это постоянная структура алгоритма, а НЕ гарантия времени Python.
        """
        if not 0 < a < modulus:
            raise ValueError('Для инверсии требуется 0 < a < modulus')
        exponent = modulus - 2  # публичная константа
        result = 1
        for i in range(modulus.bit_length() - 1, -1, -1):
            square = result * result % modulus
            product = square * a % modulus
            mask = -((exponent >> i) & 1)
            result = square ^ ((square ^ product) & mask)
        return result

# МОДУЛЬ 1. Арифметика эллиптической кривой, аффинные координаты.
@dataclass(frozen=True, slots=True)
class Point:
    x: int = 0
    y: int = 0
    infinity: bool = True

O = Point()

class Curve:
    p = int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F', 16)
    a = int('00', 16)
    b = int('07', 16)
    q = int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141', 16)
    G = Point(
        int('79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798', 16),
        int('483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8', 16), False)

    def on_curve(self, P: Point) -> bool:
        return P.infinity or (0 <= P.x < self.p and 0 <= P.y < self.p
            and (P.y * P.y - P.x ** 3 - self.a * P.x - self.b) % self.p == 0)

    def add(self, P: Point, Q: Point) -> Point:
        if P.infinity:
            return Q
        if Q.infinity:
            return P
        if P.x == Q.x:
            if P.y != Q.y or P.y == 0:
                return O
            numerator = (3 * P.x * P.x + self.a) % self.p
            denominator = 2 * P.y % self.p
        else:
            numerator = (Q.y - P.y) % self.p
```

```
            denominator = (Q.x - P.x) % self.p
        slope = numerator * Protection.inverse(denominator, self.p) % self.p
        x = (slope * slope - P.x - Q.x) % self.p
        y = (slope * (P.x - x) - P.y) % self.p
        return Point(x, y, False)

    def double_add(self, scalar: int, P: Point) -> Point:
        if scalar < 0:
            raise ValueError('Отрицательный скаляр')
        R = O
        for i in range(scalar.bit_length() - 1, -1, -1):
            R = self.add(R, R)
            if (scalar >> i) & 1:
                R = self.add(R, P)
        return R

    @staticmethod
    def cswap(A: Point, B: Point, bit: int) -> tuple[Point, Point]:
        mask = -bit
        x = (A.x ^ B.x) & mask
        y = (A.y ^ B.y) & mask
        infinity = (int(A.infinity) ^ int(B.infinity)) & mask
        return (Point(A.x ^ x, A.y ^ y, bool(int(A.infinity) ^ infinity)),
                Point(B.x ^ x, B.y ^ y, bool(int(B.infinity) ^ infinity)))

    def ladder(self, scalar: int, P: Point) -> Point:
        if not 0 <= scalar < (1 << 256):
            raise ValueError('Скаляр должен занимать не более 256 бит')
        R0, R1 = O, P
        for i in range(255, -1, -1):
            bit = (scalar >> i) & 1
            R0, R1 = self.cswap(R0, R1, bit)
            total = self.add(R0, R1)
            doubled = self.add(R0, R0)
            R0, R1 = doubled, total
            R0, R1 = self.cswap(R0, R1, bit)
        return R0

    def valid_public(self, P: Point) -> bool:
        return not P.infinity and self.on_curve(P) and self.double_add(self.q, P).infinity

# МОДУЛЬ 2. Генерация ключей, ECDH и ECDSA.
class Protocols:
    def __init__(self):
        self.curve = Curve()

    def validate_secret(self, d: int) -> None:
        if not 0 < d < self.curve.q:
            raise ValueError('Требуется 0 < секретный ключ < q')

    def random_secret(self) -> int:
        return secrets.randbelow(self.curve.q - 1) + 1

    def public_key(self, d: int) -> Point:
        self.validate_secret(d)
        return self.curve.ladder(d, self.curve.G)

    def ecdh(self, d: int, Q: Point) -> bytes:
        self.validate_secret(d)
        if not self.curve.valid_public(Q):
            raise ValueError('Недопустимый открытый ключ')
        R = self.curve.ladder(d, Q)
        if R.infinity:
            raise ValueError('Общая точка равна O')
        # Сырой секрет, для прикладного шифрования нужны KDF и аутентификация.
        return R.x.to_bytes(32, 'big')

    @staticmethod
    def digest(message_hex: str) -> int:
        if len(message_hex) % 2 or any(c not in '0123456789abcdefABCDEF' for c in message_hex):
            raise ValueError('Сообщение должно содержать чётное число hex-цифр')
        return int.from_bytes(hashlib.sha256(bytes.fromhex(message_hex)).digest(), 'big')

    def sign_hash(self, d: int, e: int) -> tuple[int, int]:
        self.validate_secret(d)
        q = self.curve.q
        while True:
            k = self.random_secret()
            C = self.curve.ladder(k, self.curve.G)
            r = C.x % q
```

```
            if r == 0:
                continue
            s = Protection.inverse(k, q) * (e + r * d) % q
            if s != 0:
                return r, s

    def verify_hash(self, Q: Point, e: int, signature: tuple[int, int]) -> bool:
        r, s = signature
        q = self.curve.q
        if not (0 < r < q and 0 < s < q) or not self.curve.valid_public(Q):
            return False
        w = Protection.inverse(s, q)
        u1, u2 = e * w % q, r * w % q
        C = self.curve.add(self.curve.double_add(u1, self.curve.G),
                           self.curve.double_add(u2, Q))
        return not C.infinity and C.x % q == r

# Hex-интерфейс и встроенные тесты.
def parse_hex(value: str) -> int:
    if not 1 <= len(value) <= 64 or any(c not in '0123456789abcdefABCDEF' for c in value):
        raise ValueError('Ожидается от 1 до 64 hex-цифр без 0x')
    return int(value, 16)

def print_point(name: str, P: Point) -> None:
    print(f'{name}.infinity={int(P.infinity)}')
    if not P.infinity:
        print(f'{name}.x={P.x:064X}\n{name}.y={P.y:064X}')

def selftest(p: Protocols) -> None:
    count = 0
    def check(condition: bool, name: str) -> None:
        nonlocal count
        if not condition:
            raise AssertionError(name)
        count += 1
        print('PASS', name)
    c = p.curve
    check(c.on_curve(c.G), 'G on curve')
    check(c.double_add(c.q, c.G).infinity, 'qG = O')
    check(c.add(c.G, O) == c.G, 'G + O = G')
    check(c.add(c.G, Point(c.G.x, -c.G.y % c.p, False)).infinity, 'G + (-G) = O')
    check(c.ladder(0, c.G).infinity, '0G = O')
    for k in range(1, 13):
        check(c.ladder(k, c.G) == c.double_add(k, c.G), f'ladder equals double-and-add k={k:X}')
    for modulus in (c.p, c.q):
        for a in (1, 2, 3, modulus - 1):
            inverse = Protection.inverse(a, modulus)
            check(a * inverse % modulus == 1 and inverse == pow(a, -1, modulus), 'Fermat inverse equals
    Python inverse')
    try:
        Protection.inverse(0, c.p)
    except ValueError:
        check(True, 'zero inverse rejected')
    else:
        check(False, 'zero inverse rejected')
    # Известные координаты 2G — независимый эталон для арифметики.
    expected = Point(
        int('C6047F9441ED7D6D3045406E95C07CD85C778E4B8CEF3CA7ABAC09B95C709EE5', 16),
        int('1AE168FEA63DC339A3C58419466CEAEEF7F632653266D0E1236431A950CFE52A', 16), False)
    check(c.add(c.G, c.G) == expected, 'known secp256k1 2G vector')
    da, db = int('123456789ABCDEF', 16), int('FEDCBA987654321', 16)
    A, B = p.public_key(da), p.public_key(db)
    sa, sb = p.ecdh(da, B), p.ecdh(db, A)
    check(sa == sb, 'ECDH equal secrets')
    check(sa.hex().upper() == 'D8EB786721FF3735E2E64B90EB05B720AA7A65EECC93E22BAB62D13580565C78', 'ECDH
    agrees with C++ version')
    print_point('QA', A)
    print_point('QB', B)
    print(f'Shared_A={sa.hex().upper()}\nShared_B={sb.hex().upper()}')
    messages = ('', '616263', '4C616234205661726961E742033')
    hashes = ('E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855',
              'BA7816BF8F01CFEA414140DE5DAE2223B00361A396177A9CB410FF61F20015AD',
              '799167108315FAE29282BA2A0A7CCCB36478A1D69D44FA5432F40E80604C52A8')
    for message, expected_hash in zip(messages, hashes):
        e = p.digest(message)
        check(e == int(expected_hash, 16), 'SHA-256 known vector')
        r, s = p.sign_hash(da, e)
        print(f'message={message}\nhash={e:064X}\nr={r:064X}\ns={s:064X}')
        check(p.verify_hash(A, e, (r, s)), 'valid signature')
        changed = bytearray(s.to_bytes(32, 'big'))
```

```
            changed[-1] ^= 1
            check(not p.verify_hash(A, e, (r, int.from_bytes(changed, 'big'))), 'one byte of s changed:
        rejected')
            check(not p.verify_hash(A, p.digest(message + '00'), (r, s)), 'changed message rejected')
            check(not p.verify_hash(B, e, (r, s)), 'wrong public key rejected')
        check(not p.verify_hash(O, 1, (1, 1)), 'infinity public key rejected')
        check(not p.verify_hash(A, 1, (0, 1)), 'r = 0 rejected')
        check(not p.verify_hash(A, 1, (1, c.q)), 's = q rejected')
        check(not c.valid_public(Point(c.G.x, 0, False)), 'off-curve key rejected')
        check(c.valid_public(p.public_key(p.random_secret())), 'random key generation')
        print(f'ALL TESTS PASSED: {count}')

    def main(args: list[str]) -> None:
        p = Protocols()
        if not args or args == ['--test']:
            selftest(p)
        elif args == ['keygen']:
            d = p.random_secret()
            print(f'd={d:064X}')
            print_point('Q', p.public_key(d))
        elif len(args) == 2 and args[0] == 'pub':
            print_point('Q', p.public_key(parse_hex(args[1])))
        elif len(args) == 4 and args[0] == 'ecdh':
            result = p.ecdh(parse_hex(args[1]), Point(parse_hex(args[2]), parse_hex(args[3]), False))
            print('shared=' + result.hex().upper())
        elif len(args) == 3 and args[0] == 'sign':
            r, s = p.sign_hash(parse_hex(args[1]), p.digest(args[2]))
            print(f'r={r:064X}\ns={s:064X}')
        elif len(args) == 6 and args[0] == 'verify':
            Q = Point(parse_hex(args[1]), parse_hex(args[2]), False)
            valid = p.verify_hash(Q, p.digest(args[3]), (parse_hex(args[4]), parse_hex(args[5])))
            print('valid=' + str(int(valid)))
        else:
            raise ValueError(__doc__)

    if __name__ == '__main__':
        try:
            main(sys.argv[1:])
        except (ValueError, AssertionError) as error:
            print(f'ERROR: {error}', file=sys.stderr)
            sys.exit(1)
```